

Evidence-Driven Validation of a Hybrid Network Intrusion Detection System: From Offline Replay to Prepared Live Capture

Azhad Shahzad Shaik 

shaikazhadshahzad@gmail.com <https://github.com/s-shahzad/Universal-NIDS>

[FILL: month year]

Abstract

Most network intrusion detection (NIDS) research reports offline accuracy on a labelled dataset and stops there. We argue that for a hybrid detector intended for real deployment, the validation process is itself a contribution. We present a hybrid NIDS that combines deterministic signature rules, statistical anomaly detection, a supervised ensemble (random forest, extra-trees, histogram gradient boosting, and XGBoost), an optional unsupervised path (isolation forest plus a shallow autoencoder), and a fusion layer that adjudicates the component outputs into a single alert decision. Alongside the detector we describe a layered, reproducible validation method: a deterministic test suite with an enforced coverage floor, repeatable offline PCAP-replay scenarios, and prepared-environment runs on live capture that measure benign false positives, queue-loss accounting, restart recovery, and sustained-soak behaviour. On a held-out test split of the labelled traffic corpus the supervised ensemble reaches **99.5% accuracy and 0.996 weighted F_1** across five classes, while the per-class results expose a genuine weakness on the rarest class that offline aggregates hide. The prepared-environment evidence shows the live path, loss accounting, operator workflows, and a completed six-hour soak (87,533 flows, 0 alerts, ≈ 400 MiB peak memory) all function as intended. We still characterise the system as a controlled pre-deployment candidate rather than a production system, because benign adjudication and platform coverage remain narrow, and we release the code, datasets, and evidence records so the validation is reproducible.

1 Introduction

Network intrusion detection is hard for two reasons that are usually addressed separately. The first is *detection quality*: distinguishing malicious from benign traffic. The second is *operational credibility*: showing that a detector behaves acceptably on live traffic, under benign drift, and through the maintenance actions an operator actually performs. Benchmark-oriented NIDS papers answer the first and rarely the second; a high offline F_1 says little about false-positive burden on live capture or about whether the system survives a restart without losing continuity.

This paper treats a hybrid NIDS *and its validation process* as one research object. The detector combines five detection paths and a fusion layer (§3). The validation method (§6) is layered: deterministic tests, repeatable offline replay, and prepared-environment live runs that measure the things offline metrics cannot, benign false positives, packet-queue loss, restart recovery, and soak stability. Our position is that this second half is where deployment confidence is earned or lost.

Contributions.

1. A hybrid NIDS architecture that retains component-level evidence for every decision, so an analyst can see why an alert did or did not fire (§3, §4).
2. A layered, reproducible validation method spanning code tests, offline replay, and prepared-environment live capture (§6).
3. An honest evaluation that reports both the strong aggregate offline result and the per-class and live-capture weaknesses it hides (§6, §7).
4. A released harness, labelled dataset split, and evidence records for reproducibility.

2 Related Work

We group prior NIDS work into four families and position ours against them.

Signature and rule engines such as Suricata and Zeek are precise and interpretable but weak on behaviour not covered by a rule; our signature engine plays the same role and is deliberately only one of five paths.

Supervised-ML IDS on labelled datasets dominate the literature and report strong offline scores. Much of this work is evaluated on the KDD Cup/NSL-KDD family [5], whose `dos/probe/r2l/u2r` attack taxonomy we adopt for labels, and on the flow-based CIC-IDS2017 corpus [6]. Ensemble and feature-selection classifiers push aggregate accuracy high on these sets [7], but the community has also documented that headline numbers on these datasets can be fragile: label and processing errors in CIC-IDS2017 materially change reported performance [8]. This is exactly why we report per-class results and treat offline accuracy as one input, not the verdict.

Anomaly and statistical IDS improve sensitivity to novel behaviour but are prone to benign-drift false positives; our unsupervised path is gated precisely to contain that failure mode.

Hybrid and validation-oriented work is closest to ours. Our own prior work applied supervised ML to denial-of-service detection on IoT EV-charging infrastructure using a CIC-IDS-derived binary corpus [9]; the present work is distinct in three ways: it is a five-path hybrid rather than a single classifier, it evaluates a five-class taxonomy rather than binary DoS/benign, and its central contribution is the prepared-environment validation method rather than the offline score. We build against operational guidance from NIST SP 800-115, NIST SP 800-61r3, MITRE ATT&CK, and CISA logging/SIEM guidance [1, 2, 3, 4].

3 System Architecture

The system ingests from live NIC capture, offline PCAP replay, optional Suricata/Zeek JSON adapters, and static artifact intake, then normalises all inputs into a common flow-event schema before feature extraction. Normalised features fan out to the detection engines and a fusion layer, and results persist to SQLite and JSONL with an analyst dashboard and reporting on top.

[FILL: include architecture diagram: export
thesis/diagrams/system_architecture.mmd to PDF (mermaid-cli) into
paper/figures/architecture.pdf, then \includegraphics here]

Figure 1: End-to-end architecture: ingest, normalise, five detection paths, fusion, suppression, persistence, and reporting.

Artifact analysis is static only; submitted files are never executed.

4 Detection Methodology

- **Signature engine.** YAML-defined rules for known behaviour; produces reviewable, deterministic alerts.
- **Statistical anomaly engine.** EWMA and z-score style thresholds with burst detection over flow aggregates.
- **Supervised ensemble.** Random forest, extra-trees, histogram gradient boosting, and XGBoost (when available), weighted into a single probability vector. **[FILL: state ensemble weighting / voting scheme from src/NIDS/ml/supervised_ensemble.py in one precise sentence.]**
- **Unsupervised path (optional).** Isolation forest plus a shallow autoencoder with warmup calibration; gated to require at least two active components before it can raise an alert, a false-positive control introduced during tuning (§6).
- **Fusion.** Combines component scores into a benign / informational / alert decision while retaining each component’s contribution as evidence.

5 Datasets and Experimental Setup

Labelled corpus. The supervised ensemble is trained and evaluated on a labelled traffic corpus of **25,000 flows** across five classes (`dos`, `normal`, `probe`, `r2l`, `u2r`), split **18,750 train / 6,250 held-out test**. The label taxonomy is the KDD-family `dos/normal/probe/r2l/u2r` scheme [5], but the features are *not* the classic 41 KDD attributes: the model consumes this system’s own 15-column flow schema (packet length, ports, protocol and TCP-flag indicators, per-window destination-port and -host counts, and DNS/HTTP/TLS presence flags) from `src/NIDS/ml/featureset.py`. The labels follow KDD conventions while the feature space is the deployed detector’s own. **[FILL: state precisely how the labelled training flows were produced or sourced (own labelled capture vs. re-mapped public corpus) and cite it; this must be exact before submission.]** We report the held-out split as the primary result; a second evaluation over the full 25,000 flows is reported for comparison and is *not* a held-out number. The class distribution is heavily imbalanced: `u2r` is so rare that it has no representative in the held-out test split, and `probe` and `r2l` have only tens of samples. We keep this visible rather than smoothing it into a single accuracy figure.

Live and offline scenarios. Offline PCAP-replay scenarios (`LAB-SCN-001..005`: port scan, HTTP brute force, flood/burst, mixed benign+malicious, artifact+network correlation) and **thirteen** prepared-environment live scenarios (`PREP-ENV-001..013`: live capture, malformed-packet handling, queue-loss accounting, restart recovery, benign soak, full-duration soak, operator workflows, and live suppression) are executed through released scripts and indexed as manifests. At draft time the index records 36 runs with all thirteen prepared-environment scenarios passing.

Measure	Value
Tests collected (public CI scope)	245
Passing (default suite)	229
Deselected (lab/environment/live markers)	16
Active pytest warnings	0
Coverage (line)	75.83%
Enforced coverage floor	72%

Table 1: Deterministic test-suite result at draft time (public CI scope). CI is green on all three workflows.

Class	Precision	Recall	F_1	Support
dos	0.999	0.996	0.997	4941
normal	0.998	0.995	0.996	1243
probe	0.725	0.980	0.833	51
r2l	0.933	0.933	0.933	15
u2r	<i>no samples in this test split</i>			
Weighted avg	0.996	0.995	0.996	6250
Accuracy	0.9954			

Table 2: Supervised ensemble on the 6,250-flow held-out test split. Aggregate accuracy is 99.5%, but the **probe** and **r2l** classes, with 51 and 15 test samples, are materially weaker (precision 0.72 on **probe**), and the **u2r** class has no samples in this split at all, so nothing can be claimed about it. These are the honest costs that a single weighted aggregate hides.

6 Results

6.1 Software validation

6.2 Supervised ensemble, held-out test split

The aggregate is strong, but Table 2 is deliberately reported per-class: the rare classes are where the ensemble is weakest, and any deployment claim must account for the small support behind their scores. We treat these as offline-evaluation numbers, not live operational accuracy.

6.3 Prepared-environment (live) results

6.4 False-positive tuning

Early prepared-environment runs raised unsupervised-only alerts on benign status polling and routine DNS. We added a control requiring at least two active unsupervised components before an alert can fire; the tuned rerun cleared the exercised benign sample (**[FILL: PREP-ENV-005: flows, 0 alerts]**). We claim this only for the exercised sample; broader benign corpora are future work.

Scenario	Result
Queue-loss accounting (PREP-ENV-003)	Under a deliberate DNS flood, 8,127 packets received, 23 processed, 8,100 dropped (99.67% loss), queue-depth peak 1, 0 false alerts. The point is that loss is <i>accounted</i> , not that it is avoided.
Benign soak, tuned (PREP-ENV-005)	1,404 flows over 180 s, 0 alerts; the two unsupervised false positives seen before tuning dropped to 0.
Restart recovery (PREP-ENV-006)	Two-phase restart, 144 flows, continuity preserved; reload latency ≈ 13.2 s.
Full-duration soak (PREP-ENV-007)	Completed 6-hour run (21,600 s), 87,533 flows, 0 alerts, 0 dropped packets, peak RSS 409,444 KiB (≈ 400 MiB), peak CPU 107%, restart latency 13.335 s.
Operator workflows	Rule refresh (PREP-ENV-008): 321 flows, 1 alert. Model swap (PREP-ENV-009): 330 flows, 2 alerts. Config override (PREP-ENV-010): 394 flows, 0 alerts. Each reload ≈ 13.2 – 13.3 s.

Table 3: Prepared-environment live-capture evidence, from NIDS_TestLab/reports/prepared_env_validation_index.json (2026-03-14, 36 recorded runs, all 13 scenarios passing). Values frozen at draft time.

7 Limitations

- **Single soak run.** The six-hour soak completed cleanly, but it is one run at one traffic profile; we do not generalise it to arbitrary deployment loads or durations.
- **Rare-class detection.** `probe/r2l` have small test support and weaker scores, and `u2r` is absent from the held-out split (Table 2).
- **Benign adjudication is narrow.** Tuning is validated on the exercised sample, not a broad benign corpus.
- **Maintenance.** Operator workflows are restart-based, implying an ≈ 13 -second blind window per reload; not zero-downtime.
- **Platform scope.** [FILL: state validated OS/interface scope honestly (Windows first-class lab host, Linux runtime target per the repo docs); confirm before submission.]

8 Conclusion

We presented a hybrid NIDS whose contribution is as much its layered, reproducible validation as its detection stack. The supervised ensemble is strong in aggregate (99.5% held-out accuracy) but honestly weaker on rare classes, and the prepared-environment evidence, including a completed six-hour live soak, supports a *controlled pre-deployment candidate* verdict rather than production readiness. We release the harness, dataset split, and evidence records so the result is reproducible.

References

- [1] NIST. *SP 800-115: Technical Guide to Information Security Testing and Assessment*. <https://csrc.nist.gov/pubs/sp/800/115/final>
- [2] NIST. *SP 800-61r3: Incident Response Recommendations*. <https://csrc.nist.gov/pubs/sp/800/61/r3/final>
- [3] MITRE. *ATT&CK Enterprise Matrix*. <https://attack.mitre.org/matrices/enterprise/>
- [4] CISA et al. *Best Practices for Event Logging and Threat Detection*. <https://www.cisa.gov/resources-tools/resources/best-practices-event-logging-and-threat-detection>
- [5] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani. A detailed analysis of the KDD CUP 99 data set. In *IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, 2009. (NSL-KDD dataset.)
- [6] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *4th Intl. Conf. on Information Systems Security and Privacy (ICISSP)*, 2018. (CIC-IDS2017 dataset.)
- [7] Y. Zhou, G. Cheng, S. Jiang, and M. Dai. Building an efficient intrusion detection system based on feature selection and ensemble classifier. *Computer Networks*, 174:107247, 2020. Preprint <https://arxiv.org/abs/1904.01352>.
- [8] G. Engelen, V. Rimmer, and W. Joosen. Troubleshooting an intrusion detection dataset: the CICIDS2017 case study. In *IEEE Security and Privacy Workshops (WTMC)*, 2021.
- [9] A. S. Shaik, M. Gutla, A. P. R. Kagitala, and S. Bhatia. Plugged-in and protected: Leveraging machine learning to secure IoT-based electric vehicle charging stations from denial-of-service threats. In *IEEE Global Conference on Artificial Intelligence and Internet of Things (GCAIoT)*, 2025. DOI: 10.1109/GCAIoT68269.2025.11275540.